

Drawbacks of using containers

1. The images from dockerhub are not always reliable or cannot be trusted completely.
 - They might have hidden issues
 - Not updated regularly
 - Misconfigurations
 - Security vulnerabilities
2. Container networking is complicated.
 - Handling port conflicts
 - Managing network types as each type behaves differently
 - Port mapping is not easy
 - Managing IP addresses
 - Handling the communication between multiple containers where as multiple services depend on each other.
3. Docker alone cannot efficiently control or coordinate many containers.
 - Docker doesn't provide auto scaling
 - traffic distribution becomes difficult
 - No automatic recovery after failure of containers
 - Docker doesn't provide rolling updates & no health checks
 - No multi-server management is provided by docker.
4. Stopped or unused containers consume lot of storage.
 - As they store metadata & logs they consume space
 - Leftover images, networks takes disk space'
 - They store layers of file system as well
 - Volumes also add up & slowly consumes lot of space unless it is manually cleaned.
5. Containers behaviour can vary across different systems.
 - Due to different docker versions
 - Due to different host operating system
 - Change in environment variable
 - Different network configurations

To overcome these drawbacks we not only go for containers alone but use container orchestration too.

Container orchestration

- >> It is the automated process/management of containers by increasing the efficiency of containers through deploying, auto-scaling, load balancing, restarting the failed or crashed containers & keeping the containers in running state so that the application runs smoothly without any manual efforts.
- >> It is the system or the process that controls or organizes everything related to the containers.

Container orchestration tools

1. Kubernetes
2. Docker Swarm
3. Rancher
4. Amazon EKS (Elastic Kubernetes Service)
5. Azure Kubernetes Service (AKS)
6. Google Kubernetes Engine (GKE)
7. Nomad (HashiCorp Nomad)
8. Apache Mesos

Kubernetes

- >> It is the free and open source container orchestration platform which is used to automate the process of monitoring, scaling, managing the containerized application & deployment.
- >> Instead of managing the multiple containers with docker we go for kubernetes,
 - It schedules the container automatically considering the RAM, CPU & health of the server.
 - Self healing (kubernetes replaces the containers that fails or crashes)
 - Auto scaling (If traffic increases - It adds up the containers)
(If traffic decreases - It deletes the extra containers)
 - Load balancing (Distributing load evenly among the healthy containers)
 - Rolling updates to avoid downtime
 - Network handling & portable in any environment.

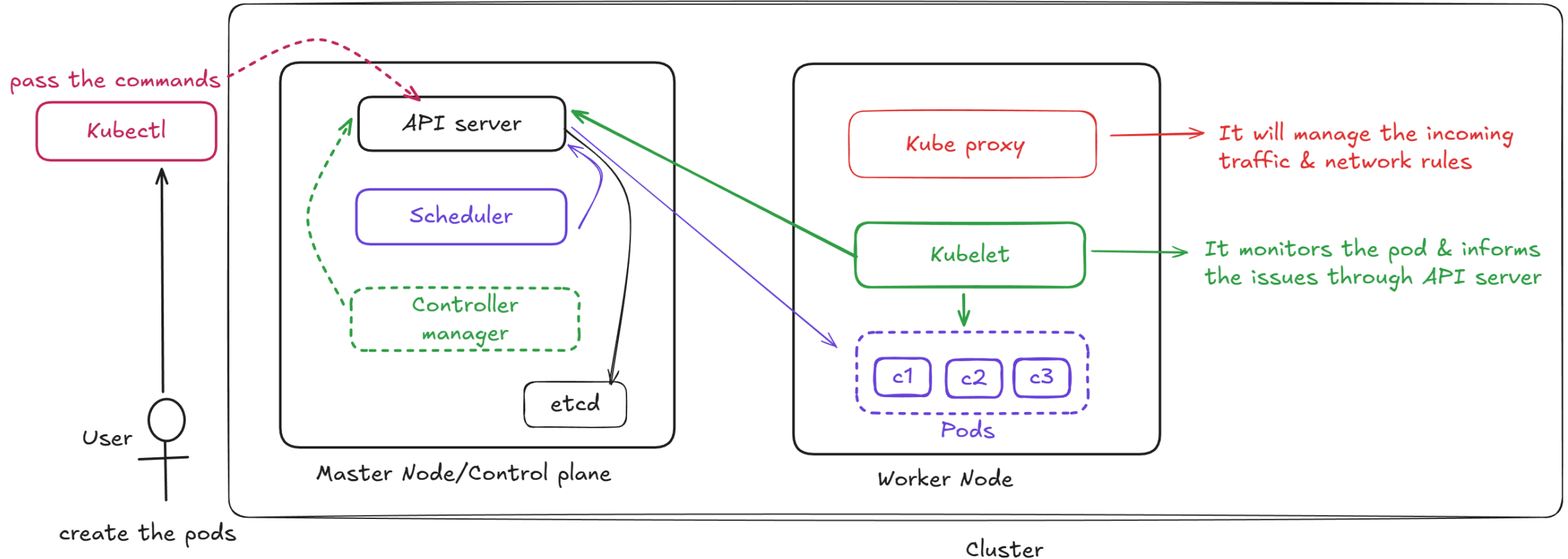
History of kubernetes

1. Google originally used an internal system called as "Borg". Borg was used to run the large scale containerised application & was powerful however was available only inside google.
2. To share this technology & make it open source they came up with the idea of kubernetes.
3. Kubernetes was officially released in the year 2015 & was made free & open source.
4. The name kubernetes comes from a greek word called as "helmsman" or "ship pilot" (symbolizes control, redirection, guidance)
5. In the year 2016, CNCF made kubernetes stable by registering it as free & open source.

Features of kubernetes

1. It is free & open source
2. It helps in efficiently orchestrating & manage the containerized application
3. Load balancing, auto-scaling & self healing.
4. It allows rolling updates to prevent downtime.
5. It provides better security & stability
6. It ensures efficient utilization of resources like CPU & RAM(memory).
7. Works well in many environments
8. Multi-server management is provided

Architecture of kubernetes



Components present in architecture of kubernetes

There are 3 layers including 2 main parts :

1. Cluster :

>> It is the collection/group of nodes that works together

- It is the combination of master node & worker node

>> It is the place where the components will be implemented & kubernetes will make the decision of where application should run.

2. Master node/control panel :

>> Master node is a decision maker that controls the workload & gives the instruction to perform the task

- It will check whether the pod is working fine or not

- Monitor the containers in pods

- Check whether the containers are healthy or not.

- Check in which specific node the pod should be created.

3. Worker node :

>> This is the place where the pods & containers will be actually running.

- Worker nodes are the virtual machines/physical server where your containers actually runs.

Components in master node and worker node

A] Master node : It contains components like,

- API server

- Scheduler

- Controller manager

- etcd

B] Worker node : It contains components like,

- Kubelet

- Kube-proxy

- Pods, container runtime.

1. API server

A gateway or endpoint of kubernetes that manages all the user request through the commands.

It forwards the request to the worker node to perform tasks

Every commands, request and configuration regarding any task goes to the API server itself.

2. Scheduler

It is used to schedule the tasks & scheduler will decide which particular worker node should run it.

- It will check the storage space(free memory)
- It will check whether the node is available or not
- It will also check whether the node is healthy or not
- It will also check for free CPU

3. Controller manager

It is responsible for managing the pods such as restart the pods if failed , scaling the pods whenever needed and if any of the node doesn't respond or work properly it will immediately shift the pod to another node.

4. etcd

It is the distributed database that stores all the details about the configurations, nodes, pods, network setup.

It stores the information in key-value pair.

5. kubectl

It is the tool or CLI used to communicate with kubernetes by passing the commands to perform certain tasks.

6. Kube proxy

It handles or manages the incoming traffic and network rules & send/forwards it to worker node.

It manages the networking within the cluster.

It makes sure that the communication between pods, services & nodes is happening smoothly.

7. Kubelet

It is present in every node & is used to monitor the pods & ensures that pods are healthy, containers are running & orders given by master is executed properly.
It informs API server if any issues occurs in pods or in nodes.

8. Pods, container runtime

Pods are the group of containers hence the application actually run inside the pods.
Container runtime is needed to run the containers in kubernetes as kubernetes doesn't run containers directly.
Ex : Docker, containerd etc

Kubernetes objects

1. Namespace
2. Pods
3. Configmap and secrets
4. Replicaset
5. Deployment
6. Services

1. Namespace

>> It is the object which is used to divide the cluster into multiple parts that helps to organize, manage the resources & isolate the application within namespaces.

veeresh	shivani	anisha	himanshi	default
- pods - deployments - configmaps - services	- pods - deployments - configmaps - services	- pods - deployments - configmaps - services	- pods - deployments - configmaps - services	- pods - deployments - configmaps - services

Cluster

Features

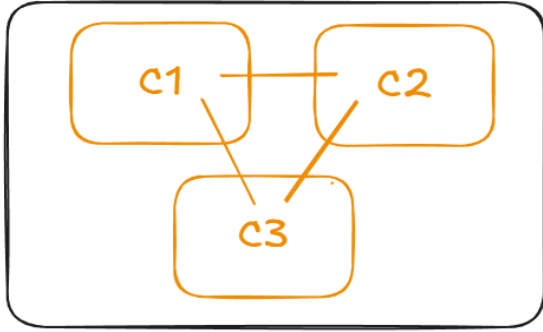
1. Same name can be allowed in different namespaces
2. Management is easy when it comes to group related work.
3. Resources inside one namespace won't be disturbing another namespace
4. For every namespace you can set the limits of resources like max memory, min CPU etc
5. Good organisation can be done here.

Commands in namespace

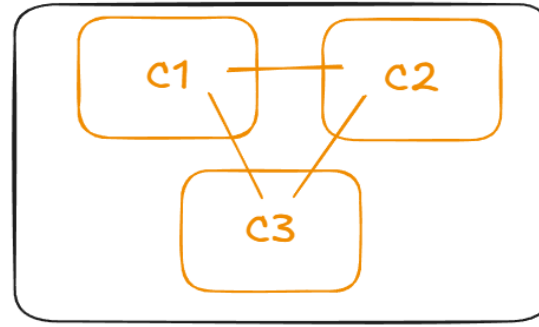
1. To create your own namespace
>> `kubectl create namespace <namespace_name>`
2. To list all namespaces (created and default namespaces)
>> `kubectl get namespace`
3. To list all the resources inside the specific namespace
>> `kubectl get all -n <namespace_name>`
4. To describe the specific namespace
>> `kubectl describe namespace <namespace_name>`
5. To delete the namespace
>> `kubectl delete namespace <namespace_name>`

2. Pods

- >> It is the group/collection of containers.
- >> It is the deployable unit in kubernetes which shares the resources within the pod among the containers that works together.
- >> Unlike docker, kubernetes doesn't handle containers directly so they require pods



POD1 - volumes



POD2 - networks

Features

1. Each pod will be having its own IP address
2. Pods can be auto restarted by deployments
3. Containers will share resources within the pod
4. Possible to increase the number of pods
5. Pods are temporary so if you restart or scale it creates new pods.

Note : 1. You can create same name pods in different namespace

Ex : N1 --> pod1

N2 --> pod1

2. It's not possible to create same name pods in same namespace

Ex : N1 --> pod1

N1 --> pod1

—————> Not possible

Basic commands of pods

1. a] To create the pod in default namespace
 >> `kubectrl run <pod_name> --image=<image_name>`

 b] To create the pod in specified namespace
 >> `kubectrl run <pod_name> --image=<image_name> -n <namespace_name>`
2. a] To list the pods in default namespace
 >> `kubectrl get pods`

 b] To list all pods in specified namespace
 >> `kubectrl get pods -n <namespace_name>`
3. a] To describe the pod in default namespace
 >> `kubectrl describe pod <pod_name>`

 b] To describe the pod in specified namespace
 >> `kubectrl describe pod <pod_name> -n <namespace_name>`
4. a] To delete the pod in default namespace
 >> `kubectrl delete pod <pod_name>`

 b] To delete all pods in default name space
 >> `kubectrl delete pod --all`

 c] To delete pod in specified namespace
 >> `kubectrl delete pod <pod_name> -n <namespace_name>`

Steps to create the pod through YAML file

Syntax :

```
apiVersion: v1
kind: Pod
metadata:
  name: <pod_name>
```

```
spec:
  containers:
    - name: <container_name>
      image: <image_name>
      ports:
        - containerPort: <port_number>

    - name: <container_name>
      image: <image_name>
```

→ container1

→ container2

Commands to execute YAML file

1. `kubectl apply -f <pod_filename>`

>> It is used to create resources from the pod YAML file

2. `kubectl port-forward <pod-name> <host_port>:<container_port>`

>> It is used to perform or implement port forwarding on pod.

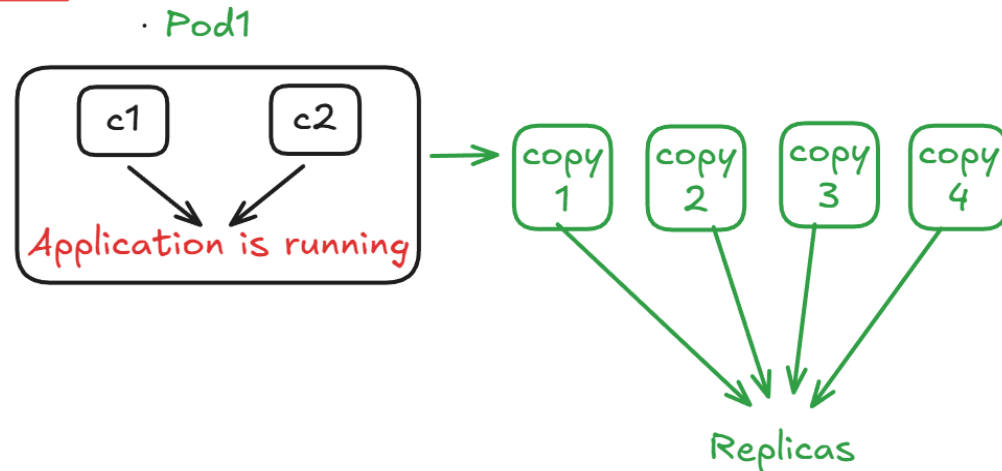
3. `kubectl exec -it <pod_name> -c <container_name> -- /bin/bash`

>> It is used to execute the commands inside the container which is present in the pod.

3. ReplicaSet

- >> ReplicaSet is the object in kubernetes that ensures desired number of identical pods are always running.
- >> RS acts like a controller to make sure that if anything goes wrong with pods then it will maintain it to keep up with desired number of pods.
 - If any pod crashes or fails it will immediately create a new one.
 - If any extra pods are present it will remove the pods
 - If scaling is needed then it will create and scale up to the desired count.

Ex :



Features of ReplicaSet

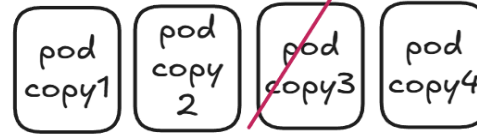
1. Self healing
2. Auto-scaling
3. Pod replacement when it crashes
4. High availability
5. Provides rolling update through deployment

Why do we go for ReplicaSet?

Scenario 1 :

Desired count of pods --> 4

Current state of pods = 4



crashed or failed



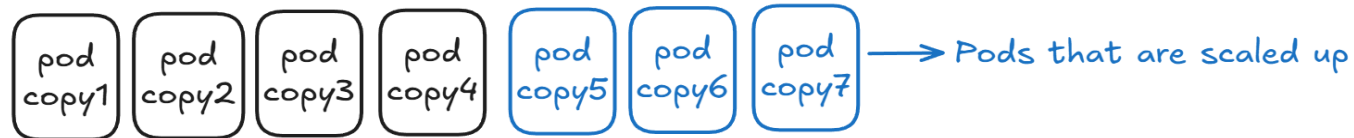
new-replaced pod

Scenario 2 :

Desired count of pod --> 4



Scale up --> 7



Scale --> 3



To go for ReplicaSet we have to follow process :

1. Labels
2. Selectors

1. Labels

- >> Labels are the tags used to identify or group the objects of kubernetes.
- >> It is the name tag attached to the objects like pods, services, replicaset & deployment.

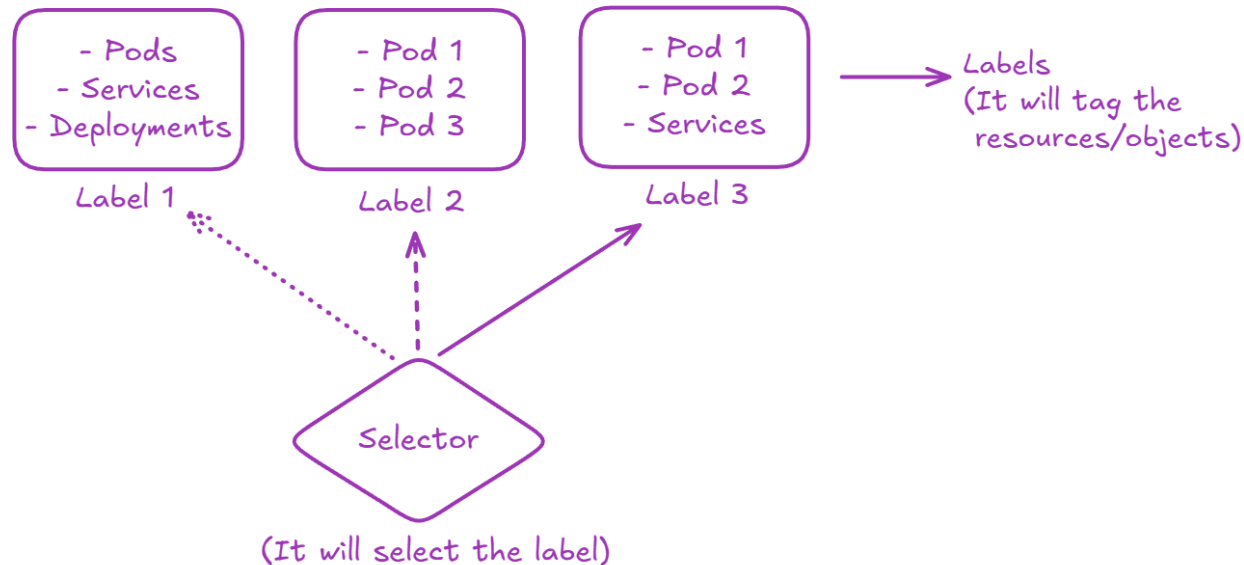
Why do we go for labels?

1. To select certain objects and group it
2. To group the pods
3. To manage application with multiple pods
4. To organize specific environment in a particular label

2. Selectors

- >> Selectors informs kubernetes exactly which set of objects to be picked based on labels.
- >> ReplicaSet makes use of selectors to decide which pod belongs to it.

Ex :



Types of selector

1. Equality based selector
2. Set based selector

1. Equality based selector

- >> Equality based selector is the default selector in kubernetes.
- >> It is the most frequently used selector that checks whether the pod has the specific label or not.
- >> In YAML file it is called as matchLabels.

How to create ReplicaSet using YAML file?

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: <name of replicaset>

spec:
  replicas: <no of replicas>
  selector:
    matchLabels:
      app: my-app

  template:
    metadata:
      name: <pod name>
      labels:
        app: my-app

    spec:
      containers:
        - name: <container name>
          image: <image name>
          ports:
            - containerPort: <port number>
```

Commands of ReplicaSet

1. To create resources from the ReplicaSet YAML file
>> `kubectl apply -f <ReplicaSet_filename>`
2. To list all ReplicaSet that is created
>> `kubectl get rs`
3. To describe ReplicaSet
>> `kubectl describe rs <ReplicaSet_name>`
4. To scale up or scale down the replicas
>> `kubectl scale rs <ReplicaSet_name> --replicas=<no of replicas>`
5. To delete the ReplicaSet along with the resources.
>> `kubectl delete rs <ReplicaSet_name>`
6. To delete the ReplicaSet without deleting the resources
>> `kubectl delete rs <ReplicaSet_name> --cascade=orphan`

Set based selector

- >> It allows kubernetes to select objects - pods, deployments or other object based on set of value.
- >> We shall use this selector depending on conditions
- >> In YAML file it is called as matchExpressions.

Types of set based selector

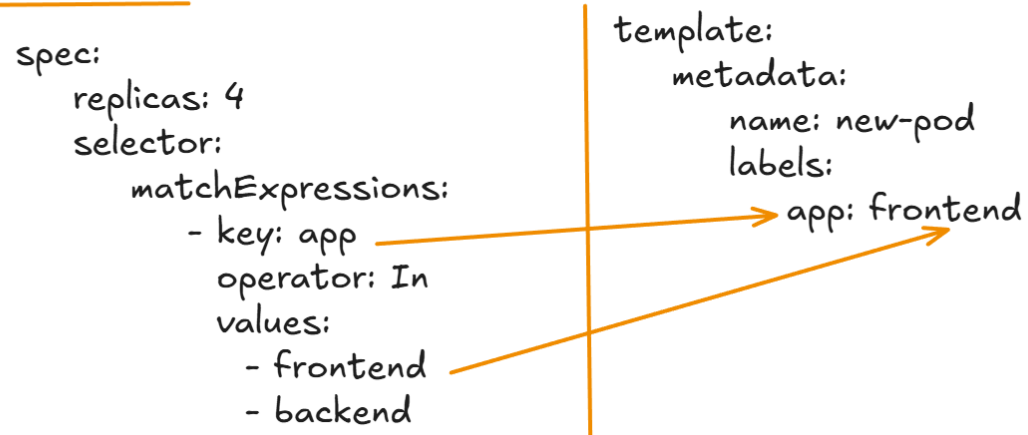
1. In
2. NotIn
3. Exist
4. DoesNotExist

1. In

>> It is used when you want to select pods whose label value matches any one value from a set of values that you have specified.

Ex : Cars in (BMW, Audi, Swift) --> swift(present in set of values)

Ex : YAML file

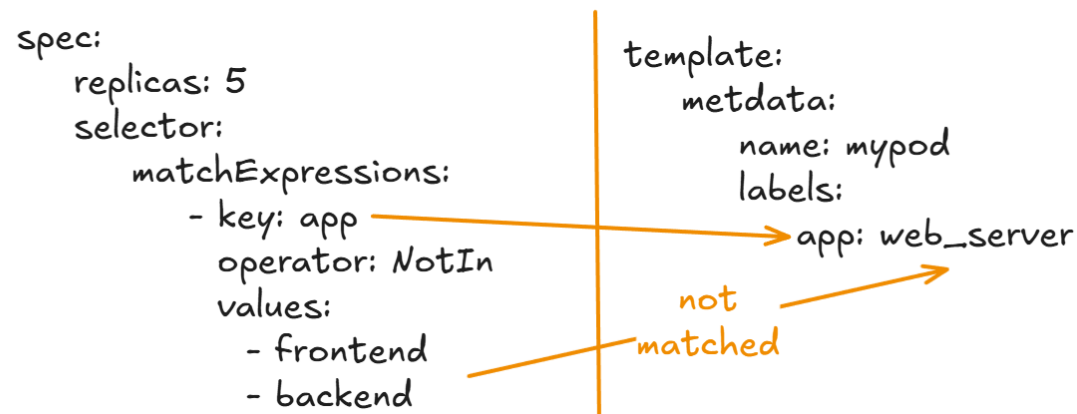


2. NotIn

>> It is used to select the pods whose label value is not present in the list of values that you have specified.

Ex : Cars notin (BMW, Audi, Swift) --> skoda(not present in set of values)

Ex : YAML file



3. Exists

>> It is used to select the pods that has a specific label key no matter what the values are.

Ex : Cars exists (skoda, BMW, Audi) --> Cars(Label key)

Ex : YAML file

<pre>spec: replicas: 5 selector: matchExpressions: - key: app operator: Exists</pre>		<pre>template: metadata: name: mypod labels: app: web_server</pre>
--	--	--



4. DoesNotExist

>> It is used to select pod that doesn't have specific label key.

Ex : YAML file

<pre>spec: replicas: 5 selector: matchExpressions: - key: app operator: DoesNotExist</pre>		<pre>template: metadata: name: mypod labels: sample: web_server</pre>
--	--	---



How to create ReplicaSet using YAML file (Set based selector)

IN :

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: setbasedreplica
spec:
  replicas: 5
  selector:
    matchExpressions:
      - key: app
        operator: In
        values:
          - frontend
          - backend
  template:
    metadata:
      name: pod12
      labels:
        app: frontend
    spec:
      containers:
        - name: nginxcont1
          image: nginx
          ports:
            - containerPort: 80
```

NOTIN :

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: setbasedreplica
spec:
  replicas: 5
  selector:
    matchExpressions:
      - key: app
        operator: NotIn
        values:
          - frontend
          - backend
  template:
    metadata:
      name: pod12
      labels:
        app: database
    spec:
      containers:
        - name: nginxcont1
          image: nginx
          ports:
            - containerPort: 80
```

EXISTS :

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: setbasedreplica
spec:
  replicas: 5
  selector:
    matchExpressions:
      - key: app
        operator: Exists
  template:
    metadata:
      name: pod12
      labels:
        app: frontend
    spec:
      containers:
        - name: nginxcont1
          image: nginx
          ports:
            - containerPort: 80
```

DOESNOTEXIST :

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: setbasedreplica
spec:
  replicas: 5
  selector:
    matchExpressions:
      - key: app
        operator: DoesNotExist
  template:
    metadata:
      name: pod12
      labels:
        sample: frontend
    spec:
      containers:
        - name: nginxcont1
          image: nginx
          ports:
            - containerPort: 80
```

Deployments

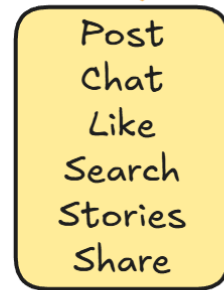
- >> It is the object in kubernetes that tells kubernetes how many pods should be created through replicaset & it makes sure that those many pods will be in a running state.
- >> It ensures the update in application happens securely without downtime.

Features of deployments

1. Rolling update
 - If updates fail then it can roll back to previous version
 - Zero downtime
2. Self healing
3. High availability of resources
4. Scalability
5. ReplicaSet management
6. Regular health check

Ex:

Instagram



v1



New updates

-
- A green rounded rectangle containing a list of updates: - Repost, - Reels, - Location share, - Music notes, - 2x speed reels, - Video editing, - Download, - Edit message.
- Repost
 - Reels
 - Location share
 - Music notes
 - 2x speed reels
 - Video editing
 - Download
 - Edit message

v2

Deployment strategies

1. Rolling update deployment
2. Blue Green deployment
3. Canary deployment

1. Rolling update deployment

- >> It is used to update the application in the pods one by one and not all at once.
- >> It will make sure that both old & new version are running together so that if any update doesn't work properly, it will roll back to older version without stopping the application to avoid downtime.

Features

1. Built-in in kubernetes
2. Zero downtime
3. Easy to understand

Commands of deployment

1. To create the resources from deployment YAML file

>> `kubectl apply -f <YAML_file_name>`

2. To list all the created deployments

>> `kubectl get deploy`

3. To describe the deployment

>> `kubectl describe deploy <deployment_name>`

4. To delete the deployment

>> `kubectl delete deploy <deployment_name>`

5. To update the container image from deployment

>> `kubectl set image deployment <deployment_name> <container_name>=<updated_image_name>`

6. To rollback to the previous version of the deployment

>> `kubectl rollout undo deployment <deployment_name>`

Steps to create the deployment through YAML file

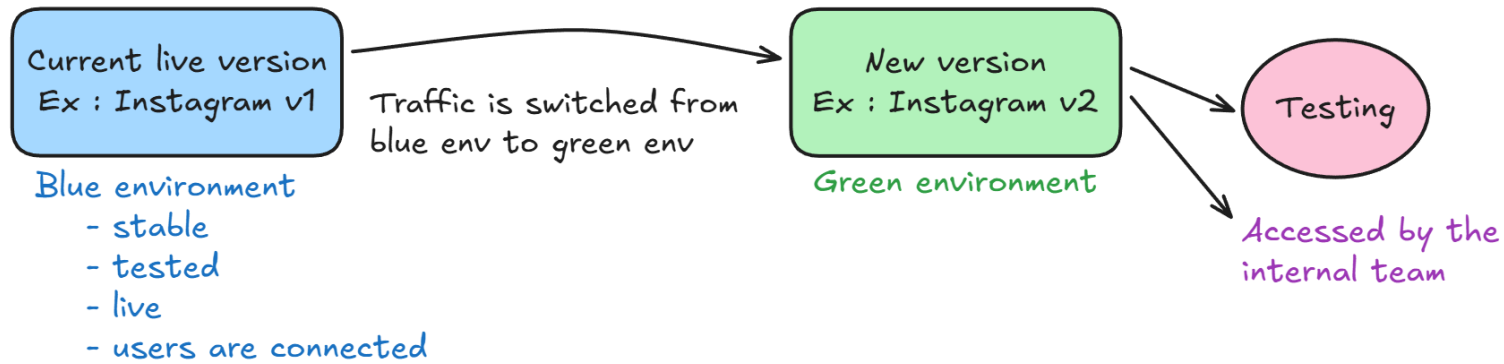
```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: deployment1
  labels:
    app: nginx
```

```
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
```

```
template:
  metadata:
    labels:
      app: nginx
```

```
spec:
  containers:
    - name: nginx
      image: nginx:1.14.2
      ports:
        - containerPort: 80
```

2. Blue Green deployment



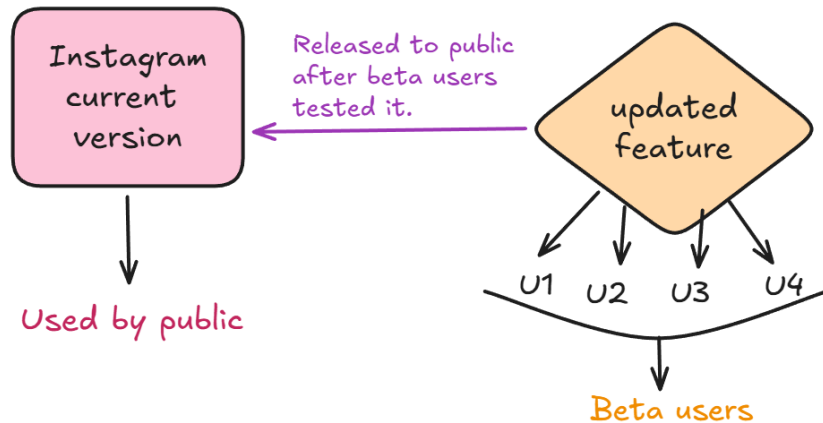
>> Blue-Green deployment maintains two environments that is blue environment and green environment where blue environment has current live version & green environment is for new version(updated features).

>> The traffic is switched only after the updated version is fully tested and stable.

Features of Blue-Green deployment

1. Immediate rollback
2. Zero downtime
3. Secure test
4. For major releases

3. Canary deployment



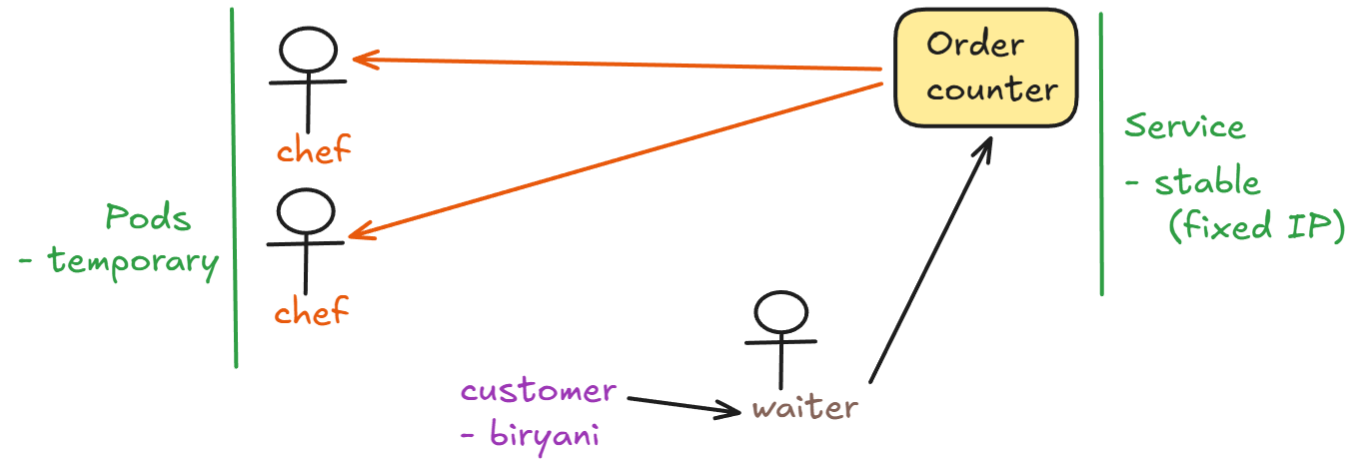
>> Canary deployment is used to release new features to small set of users first & these users are called as beta users.

>> If all features used by beta users work well, then it will be released to everyone.

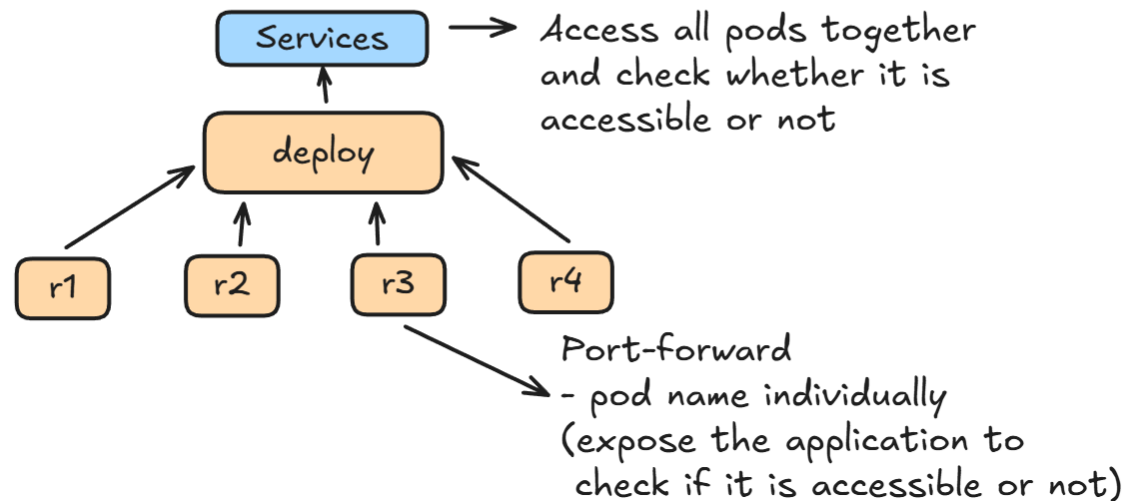
Services

- >> It is used to access the group of pods.
- >> It provides with stable IP & DNS name to access the set of pods.

S1 : Ex : Restaurant



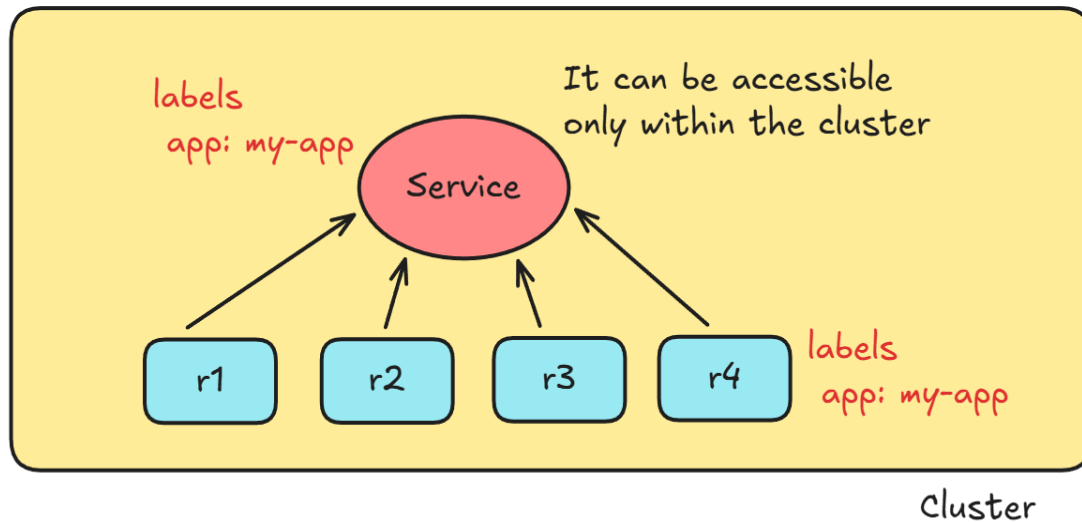
S2 : Ex :



Types of services

1. ClusterIP
2. NodePort
3. LoadBalancer

1. ClusterIP



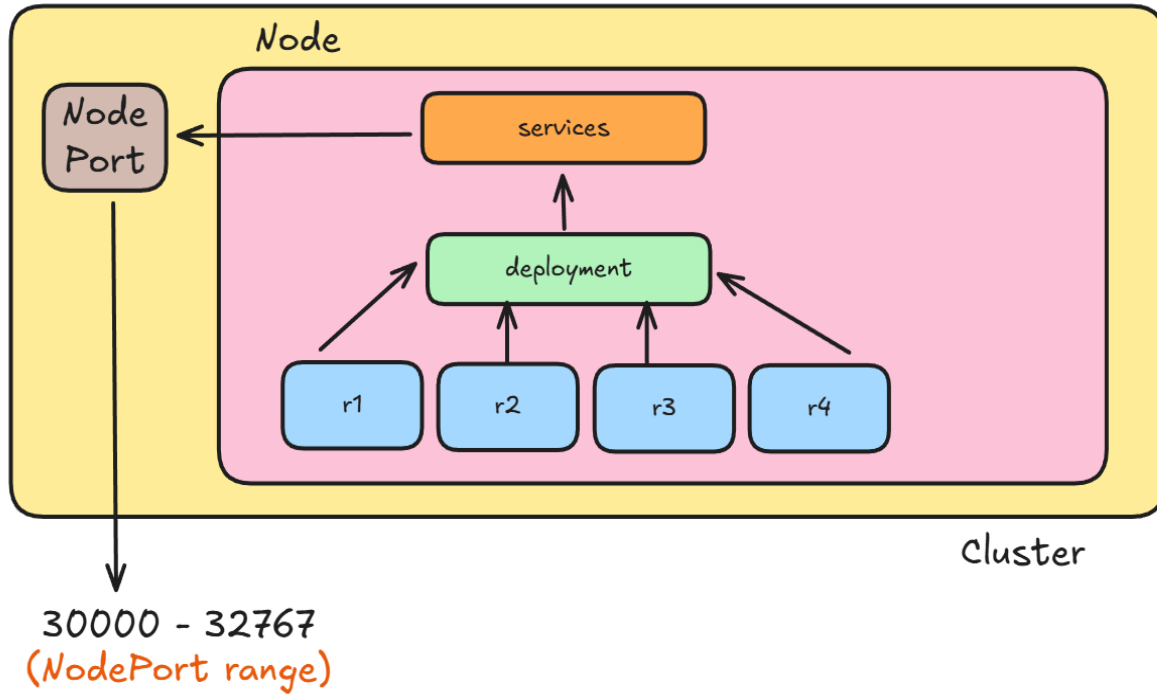
- >> ClusterIP exposes an application only inside the kubernetes cluster.
- >> It can be used by pods only within the cluster however not from outside.

clusterIP YAML file

```
apiVersion: v1
kind: Service
metadata:
  name: clusteripservice
```

```
spec:
  selector:
    app: nginx          # label(app: nginx) of deployment is
                        # matching with service
  ports:
    - port: 124         # service port
      targetPort: 80    # container-port number in deployment
```

2. NodePort



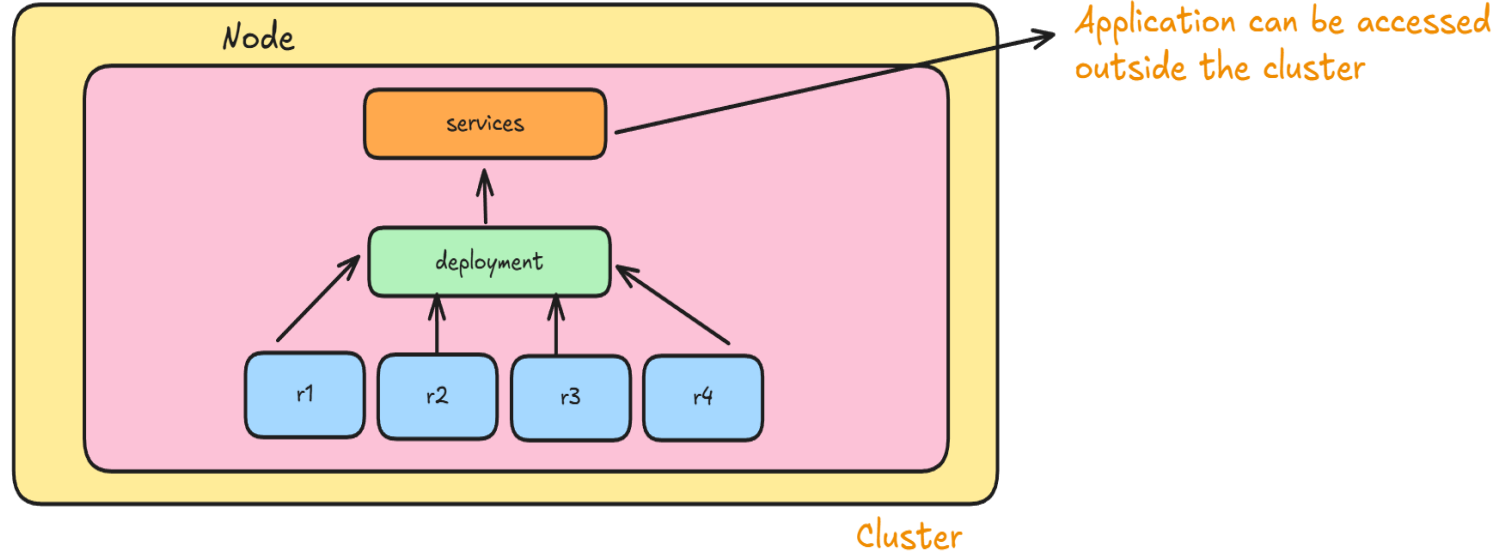
- >> NodePort exposes an application inside the kubernetes cluster as well as outside the cluster with the help of fixed port on every node.
- >> Using `NodeIP:NodePort` we can access the application.

NodePort YAML file

```
apiVersion: v1
kind: Service
metadata:
  name: nodeportservice
```

```
spec:
  type: NodePort
  selector:
    app: nginx
  ports:
    - port: 163          #service port
      targetPort: 80     # deployment container port
      nodePort: 30007
```

3. Load balancer



Steps to set up a Load Balancer in Amazon EKS (Elastic Kubernetes Service)

1. Create a new IAM user in the AWS console.
2. Log in using the newly created IAM user then create an IAM role and attach the following managed policies :
 - AmazonEKSClusterPolicy
 - AmazonEC2ContainerRegistryReadOnly
 - AmazonVPCFullAccess
 - AmazonWorkerNodePolicy
 - AmazonEKSClusterServicePolicy
 - AmazonEKS_CNI_Policy
 - AdministratorAccess
3. Launch an EC2 instance using an instance type such as t3.medium or the instance type similar to t3.medium as per the availability in region.
4. Associate the IAM role you created with the EC2 instance.
5. Connect to the EC2 instance and install the required command-line tools such as : AWS CLI, eksctl & kubectl

6. Create an EKS cluster using a script

```
Script : eksctl create cluster \  
  --name <cluster_name> \  
  --region <aws_region> \  
  --nodegroup-name <nodegroup_name> \  
  --node-type <ec2_instance_type> \  
  --nodes <desired_node_count> \  
  --nodes-min <minimum_nodes> \  
  --nodes-max <maximum_nodes> \  
  --managed
```

7. Create the YAML configuration files that is use the nano editor to create the kubernetes files & Copy the respective YAML content into each file, save them.

1) nano deployment.yaml

```
Ex : Script :- apiVersion: apps/v1  
  kind: Deployment  
  metadata:  
    name: new-deployment  
  labels:  
    app: nginx  
  spec:  
    replicas: 4  
    selector:  
      matchLabels:  
        app: nginx  
  template:  
    metadata:  
      labels:  
        app: nginx  
    spec:  
      containers:  
        - name: nginx-con1  
          image: nginx  
          ports:  
            - containerPort: 80
```

2) nano loadbalancer.yaml

Ex : Script :-

```
apiVersion: v1
```

```
kind: Service
```

```
metadata:
```

```
  name: newloadbalancer
```

```
spec:
```

```
  type: LoadBalancer
```

```
  selector:
```

```
    app: nginx          # label(app: nginx) of deployment is matching with service
```

```
  ports:
```

```
    - port: 212          #service port
```

```
      targetPort: 80    #container-port number in deployment
```

8. Execute the files to create the resources and list them to verify

a] To create

- kubectl apply -f deployment.yaml
- kubectl apply -f loadbalancer.yaml

b] To list

- kubectl get deploy
- kubectl get pods
- kubectl get svc

9. Copy the External-IP and paste it followed by service port number

Ex : <http://ac3ca75f5cbaa786bf6b6b765c4c7c278e-4534108063.us-west-2.elb.amazonaws.com:212/>

10. Delete the EKS cluster in the specified region using the eksctl delete cluster command.

Command : `eksctl delete cluster --name <cluster_name> --region <region_name>`